UNIVERSIDAD NACIONAL DE COSTA RICA

Estructuras de Datos · II Ciclo 2026

CRIPTA

Decisiones de diseño

Justificación técnica de los problemas §4.1–§4.11

Estructuras implementadas · Operaciones reales · Evidencia empírica

DOCUMENTO TÉCNICO	ALCANCE
Proyecto	Juego Cripta
Base documental	estructuras_utilizadas.md y enunciado del proyecto
Verificación	Código local y ejecución de main.py --bench
Fecha	18 de octubre de 2026

Las decisiones se justifican por su relación con las operaciones ejecutadas, las restricciones del juego y las mediciones; no únicamente por su notación asintótica.

Contenido

Este documento presenta una decisión por cada problema resuelto del §4. Todas las secciones contienen exactamente los seis apartados solicitados: problema, alternativas consideradas, decisión, análisis de complejidad, evidencia empírica y límites conocidos.

SECCIÓN	PROBLEMA DE DISEÑO	PÁG.
4.1	Qué ocurre a continuación	4
4.2	Traer el contenido antes de que haga falta	5
4.3	Resolución de referencias del catálogo	6
4.4	Reutilización y caché persistente	7
4.5	Inventario	8
4.6	Pergaminos de retroceso e historial reversible	9
4.7	El rastro del jugador	10
4.8	Bitácora en pantalla	11
4.9	Ordenamiento	12
4.10	Guardado binario de la partida	13
4.11	Bitácora de partida y reproducción	14

Metodología de medición

1. Ejecución y entorno

Se adopta la ejecución estándar python main.py --bench registrada en docs/mediciones_entrega.json: Windows 11 y Python 3.14.0. El comando incluye los proveedores originales B, los complementarios E y los perfiles P de benchmarks/bench_ampliado.py. Las tablas y los tiempos citados se actualizan con esa misma ejecución; no se mezclan con los tiempos históricos de WSL.

PARÁMETRO	CONFIGURACIÓN
Python	3.14.0 (Windows, AMD64)
Sistema	Windows 11 (10.0.26200); CPU AMD64 Family 25 Model 68
Temporización	perf_counter_ns; agenda y ciclo: perf_counter
Repeticiones	15 muestras y 1 calentamiento por caso, salvo excepciones indicadas
Ciclo completo	200 muestras; 10 calentamientos; semilla 7
Entradas generales	n = 8, 16, 32, 64, 128 y 512; semilla 123
Resumen estadístico	Mediana; microsegundos (µs) o milisegundos (ms) por lote
Cobertura de las series	633 registros: 607 B, 21 E y 5 P; todos incluidos en --bench

2. Aislamiento y comparabilidad

La preparación queda fuera del cronómetro. Se usan datos offline y archivos temporales; las lecturas pueden beneficiarse de la caché del sistema operativo y no representan disco frío. Los tiempos corresponden a Windows 11, no incluyen sincronización forzada con fsync ni garantizan el mismo rendimiento en otro equipo.

3. Tres clases de evidencia

B: proveedor original incluido en python main.py --bench. E: proveedor complementario, también incluido en ese comando. P: perfil instrumentado incluido en el mismo comando, separado del cronómetro. Los arranques parten de caché y almacenamiento temporal vacíos. desde_red identifica la fuente de datos; en estas pruebas offline no hubo HTTP. E/P ya no requieren un ejecutor externo.

4. Alcance de las conclusiones

Se comparan operaciones equivalentes cuando el benchmark lo permite. Si una alternativa omite trabajo de la solución, o si solo se mide el parseo, se declara expresamente. Los ~400 ms por solicitud proceden del enunciado: no son una medición de red propia y no se usan como evidencia experimental de latencia.

Comprobación adicional: 64 pruebas dirigidas de BFS, carga gradual, benchmarks del bloque de datos y restricciones del enunciado finalizaron correctamente. Las pruebas de corrección complementan, pero no sustituyen, los tiempos de --bench.

4.1 Qué ocurre a continuación

1. Problema

El motor debe escoger el evento más próximo y desempatar por secuencia, además de cancelar y reajustar eventos. En el ciclo demo instrumentado se observaron 4 consultas al mínimo, 2 extracciones, 2 programaciones y 5 búsquedas por ID; hubo 0 cancelaciones y 0 reprogramaciones. No se infiere de este escenario la frecuencia de esas operaciones excepcionales.

2. Alternativas consideradas

Una lista sin ordenar simplifica insertar, pero vuelve lineal cada selección del mínimo. Una lista ordenada facilita consultar un extremo, a costa de desplazar elementos al programar. Un AVL por prioridad es viable; se prefirió el montículo por su consulta directa a la raíz. La cancelación perezosa se descartó porque acumula eventos inválidos y concentra limpieza en extracciones posteriores.

3. Decisión

Se utiliza MonticuloMinimo indexado, con prioridad (tiempo, secuencia), y tres AVL IndiceOrdenado: por ID, prioridad y destinatario. Cada evento conserva su posición en el montículo. La agenda centraliza los cambios para mantener los cuatro componentes sincronizados; los efectos conservan los IDs de sus eventos.

4. Análisis de complejidad

Con n eventos, consultar cuesta O(1); programar, extraer, cancelar por ID o reprogramar cuestan O(log n), incluidos los índices. Cancelar los k eventos de un actor cuesta O(log n + k log n), sin revisar los de otros actores. La memoria es O(n). Se prioriza la consulta repetida y se paga la actualización de índices en las mutaciones.

5. Evidencia empírica

EVENTOS EXTRAÍDOS	MEDIANA DEL LOTE	TIEMPO / EVENTO
32	448,30 µs	14,01 µs
128	2.674,60 µs	20,90 µs
512	14.508,80 µs	28,34 µs

[B/P] La extracción mide la agenda completa, no solo el montículo. El ciclo demo tuvo mediana 0,13 ms; no hubo ciclos de 20 ms o más entre las 200 muestras. Esto valida ese escenario, no cualquier cripta o combinación de efectos.

6. Límites conocidos

Cuatro estructuras aumentan memoria y constantes. Millones de eventos o muchas mutaciones por consulta exigirían medir otras agendas e índices. La cancelación masiva sigue dependiendo de k; las pruebas de extracción no caracterizan cancelaciones, empates intensivos ni reajustes de velocidad.

Trazabilidad: logica/agenda_eventos.py; estructuras/monticulo_minimo.py; estructuras/indice_ordenado.py.

4.2 Traer el contenido antes de que haga falta

1. Problema

Se debe conocer el contenido antes de entrar a una sala sin gastar una solicitud por puerta. En arranques offline fríos de criptas con 6, 20, 40 y 50 salas se contaron respectivamente 1, 2, 4 y 5 lotes de contenido, con todas las salas cargadas. En la prueba de 900 salas, una planificación recorrió 900 vértices y 3.480 puertas para elegir 10 salas.

2. Alternativas consideradas

Cargar siempre todo evita esperas, pero puede agotar el presupuesto antes de jugar. Cargar solo el destino desaprovecha los lotes de hasta 10. La matriz de adyacencia ocupa $O(V^2)$ sin beneficiar un mapa disperso. Usar listas para pertenencia en BFS es sencillo, pero repite búsquedas lineales; se midió directamente esa alternativa.

3. Decisión

Grafo por adyacencia y referencias a salas vecinas; marca contenido_cargado; BFS con tablas hash propias y ordenación por (distancia, ID). Si existe holgura se carga todo; en otro caso se anticipan lotes cercanos y se reserva presupuesto. asegurar() precede al movimiento. Las respuestas se incorporan en orden de solicitud, una vez resueltas sus fichas.

4. Análisis de complejidad

El recorrido es O(V + E) esperado; ordenar las distancias añade O(V log V), de modo que la planificación completa no es solo O(V + E). La memoria auxiliar es O(V). Obtener una sala por ID sigue siendo O(V); incorporar varias salas añade ese costo además de decodificar su contenido. La red no avanza el reloj virtual.

5. Evidencia empírica

SALAS	BFS + HASH	BFS + LISTAS
100	0,60 ms	0,35 ms
400	2,52 ms	5,05 ms
900	9,57 ms	24,97 ms

[E/P] Ambos métodos devolvieron el mismo lote. El resultado pequeño puede favorecer las listas por sus constantes; al crecer el mapa se justifica evitar las comprobaciones lineales. Se midió CPU local, no descarga ni latencia HTTP.

6. Límites conocidos

El BFS actual recorre toda la componente aunque solo se pidan 10 salas. En mapas enormes convendrían cortes por nivel y un índice de salas. La carga síncrona todavía puede hacer esperar si no se anticipó el destino; sin presupuesto, se rechaza entrar a una sala pendiente.

Trazabilidad: service/cargador_contenido.py; service/juego_service.py; logica/mapa_cripta.py.

4.3 Resolución de referencias del catálogo

1. Problema

Los tipos de enemigos, objetos y trampas deben resolverse antes de usar sus estadísticas, sin solicitar fichas repetidas. Los cuatro arranques offline fríos necesitaron 11 tipos únicos y 2 lotes de catálogo, de tamaños 10 y 1. En el caso sintético de 128 tipos se contaron 128 resoluciones y 13 llamadas de precarga; no fueron solicitudes HTTP.

2. Alternativas consideradas

Solicitar un ID cada vez desperdicia el presupuesto cuando el API acepta 10. Copiar fichas dentro de cada instancia duplica información y permite retener fichas fuera del límite de memoria. Un catálogo hash sin desalojo localizaría rápido, pero también anularía --cache-size. Se prefiere compartir referencias por tipo y recuperar por niveles.

3. Decisión

Se reúnen IDs, se eliminan repeticiones y se ordenan antes de formar lotes de hasta 10. _resolver_fichas añade transitivamente el botín de enemigos. CatalogoVivo no retiene fichas: delega en una capa de acceso al catálogo, que consulta caché, disco versionado y finalmente fuente. Las entidades mantienen tipo_ficha_id y resuelven la propiedad ficha.

4. Análisis de complejidad

Para u IDs únicos, ordenar cuesta O(u log u), pero deduplicar con listas y comprobar vistos puede alcanzar $O(u^2)$. Cada acierto en caché cuesta O(log c). Con lotes de tamaño máximo 10, las búsquedas cruzadas dentro de una respuesta tienen factor acotado; la cantidad de tipos faltantes, las rondas de botín y la persistencia condicionan el costo total.

5. Evidencia empírica

TIPOS	LOTES CONTADOS	RESOLUCIÓN LOCAL
32	4	80,60 µs
128	13	399,40 µs
512	52	1.606,30 µs

[E/P] Se ejecutó el método real con un catálogo instrumentado en memoria y fichas sin botín; la preparación quedó fuera del tiempo. La reducción de 128 consultas individuales posibles a 13 precargas es un conteo de agrupación, no una medición de ahorro de latencia. El perfil de arranque sí usa las fichas reales del paquete offline.

6. Límites conocidos

Muchos IDs distintos vuelven costosas las listas de vistos: convendría lista para orden y tabla hash para pertenencia. El caso sintético no caracteriza cadenas largas de botín. Si la red falla, la capa de acceso al catálogo puede usar una versión anterior del disco; es disponibilidad con riesgo de desactualización, no equivalencia garantizada de versiones.

Trazabilidad: service/cargador_contenido.py; service/catalogo_vivo.py.

4.4 Reutilización y caché persistente

1. Problema

La consulta de fichas se repite aun sin descargarlas. En el arranque frío de cripta-04 hubo 243 consultas de caché: 232 aciertos y 11 fallos, con 11 fichas retenidas, capacidad 25 y 3 tipos fijados. El catálogo se obtuvo en lotes de 10 y 1. En los cuatro arranques hubo 83, 126, 213 y 243 consultas: muchas más que los 2 lotes de catálogo por arranque.

2. Alternativas consideradas

Una lista de búsqueda secuencial es simple y se comparó directamente. FIFO puede expulsar una ficha antigua pero usada con frecuencia; LFU exige contadores y arrastra popularidad de otras criptas. Solo disco agrega I/O a cada consulta; una caché sin límite incumple el tope. La tabla hash se excluyó por la restricción específica documentada para la caché.

3. Decisión

LRU con lista doble y arreglo ordenado (ID, nodo), localizado por búsqueda binaria. Las fijaciones por dueño protegen fichas de inventario y sala. La persistencia usa ID y versión; las entidades no retienen fichas. Se desaloja la menos reciente no fijada. Si todas están fijadas, se rechaza solo la inserción en memoria: una ficha disponible en disco se devuelve igualmente al solicitante, sin quedar retenida en caché. Ese rechazo no implica que el juego falle.

4. Análisis de complejidad

El acierto cuesta O(log c) y mover al frente O(1). Insertar o eliminar del arreglo cuesta O(c). Revisar hasta c víctimas y buscar cada fijación cuesta O(c log(f + 1) + c), con f tipos fijados. Memoria O(c + f), además del disco. El límite cuenta fichas, no bytes, y no limita por sí solo el registro de fijaciones.

5. Evidencia empírica

BÚSQUEDAS PRESENTES	LRU + BÚSQUEDA BINARIA	LISTA SECUENCIAL
32	44,00 µs	29,20 µs
128	201,80 µs	398,80 µs
512	1.028,10 µs	5.593,00 µs

[B/P] La alternativa solo busca; la caché también actualiza LRU y métricas, por lo que no son costos funcionales idénticos. Recuperar 128 fichas desalojadas desde disco tomó 46,67 ms y la fuente de prueba rechazaba cualquier acceso a red.

6. Límites conocidos

Si los tipos en uso superan la capacidad, no todos podrán permanecer en memoria: habrá recuperaciones repetidas. El benchmark no compara políticas LRU/FIFO. Capacidades grandes justifican AVL; fallos de escritura silenciados pueden impedir recuperar sin red una ficha desalojada.

Trazabilidad: logica/cache_catalogo.py; datos/almacen_persistente.py; dto/ficha_resoluble.py.

4.5 Inventario

1. Problema

El jugador navega en ambos sentidos y actúa sobre el objeto seleccionado; no necesita principalmente acceso numérico. Se midieron inserciones y recorridos de 8 a 512 objetos; cada prueba de retiro y de movimiento al frente ejecuta una operación. El proveedor adicional retiró una selección en las posiciones 0, 256 y 511 de un inventario de 512. La capacidad real de las cuatro criptas observadas fue 10.

2. Alternativas consideradas

Una lista de Python permite índices directos, pero retirar una posición interior o moverla al frente desplaza referencias. Una lista simple obliga a buscar el anterior para retroceder. Una cola doble favorece extremos, no un nodo interior. Mantener tres índices permanentes para las vistas exige sincronizarlos en cada mutación; se descartó para un inventario pequeño y acotado.

3. Decisión

ListaDobleImpl con cursor al nodo, contador y capacidad. Retirar y equipar reutilizan el nodo conocido; equipar lo mueve al frente. Las vistas por peso, valor o nombre ordenan una copia de referencias mediante OrdenadorAdaptativo, sin alterar enlaces ni cursor. Los retiros reversibles guardan identidad y marcas de orden.

4. Análisis de complejidad

Navegar, insertar, retirar el actual y moverlo al frente son O(1) estructural. Recorrer o copiar cuesta O(i); ordenar una vista, O(i log i) en el peor caso. La acción completa de servicio no es siempre O(1): resolver un objetivo y localizar su nodo pueden recorrer O(i), igual que sincronizar referencias. Restaurar retiros también puede buscar su posición en O(i).

5. Evidencia empírica

POSICIÓN SELECCIONADA	RETIRO · 512 OBJETOS
0	2,80 µs
256	2,70 µs
511	1,10 µs

[B/E] Como referencia, quitar el frente de una lista Python de 512 elementos tomó 0,30 µs. Puede ser más rápido por su implementación en C; no demuestra ausencia de desplazamientos ni satisface la exigencia estructural. El cursor se posicionó antes de cronometrar el retiro.

6. Límites conocidos

La navegación por ID o las búsquedas aleatorias no son eficientes. Con inventarios muy grandes o muchas selecciones por ID convendría un índice auxiliar ID → nodo. El benchmark de retiro aísla la estructura: no debe usarse como tiempo de soltar un objeto de extremo a extremo.

Trazabilidad: logica/inventario.py; logica/servicio_inventario.py; service/juego_service.py.

4.6 Pergaminos de retroceso e historial reversible

1. Problema

Deshacer debe revertir la decisión y todos sus eventos posteriores, sin copiar la partida ni devolver pergaminos. En el ciclo demo se contaron 20 registros de cambios, una apertura y un cierre de intervalo. El benchmark procesa de 8 a 512 intervalos con un cambio cada uno, conserva como máximo 5 y, por separado, revierte una transacción de n cambios.

2. Alternativas consideradas

Una instantánea completa por acción está prohibida y copia datos ajenos al cambio. Reproducir todas las acciones previas hace que deshacer dependa de la duración de la partida. Una pila global sin transacciones pierde los límites entre decisiones y eventos. Guardar índices de listas es inseguro si recoger o consumir pergaminos cambia posiciones de forma irreversible.

3. Decisión

Lista doble de hasta 5 transacciones y pila de inversos dentro de cada una. Cada cambio conserva datos mínimos y referencias estables. Antes de aplicar inversos se valida sobre vistas superficiales de las colecciones afectadas, sin clonar entidades ni EstadoPartida. Los pergaminos se consumen fuera del historial; si la reversión falla se devuelve el consumido.

4. Análisis de complejidad

Enlazar un registro y manipular los extremos cuesta O(1), excluido capturar su contenido. Recorrer k inversos es O(k), pero deshacer completo suma validación, búsquedas por identidad y actualizaciones de agenda O(log n). Copiar vistas o buscar posiciones puede costar O(m) por colección/operación. La memoria depende de los cambios de 5 intervalos, no es constante en bytes.

5. Evidencia empírica

n	REGISTRAR n INTERVALOS	SNAPSHOTS ALTERNATIVOS	REVERTIR n CAMBIOS
32	0,08 ms	0,23 ms	0,02 ms
128	0,32 ms	2,54 ms	0,06 ms
512	1,29 ms	37,96 ms	0,24 ms

[B/P] La alternativa medida usa deepcopy sobre un estado sintético con n referencias; ilustra el costo de copiar estado creciente, no una comparación idéntica con todo el juego. La reversión medida solo restaura vida: no caracteriza la validación de agenda o inventario. Además del tiempo, la prohibición de instantáneas decide el descarte.

6. Límites conocidos

Una acción con muchísimos eventos puede generar muchos deltas aunque solo se retengan 5 acciones. Toda mutación debe registrar su inverso; una omisión rompe la fidelidad. Validaciones sobre listas grandes pueden superar el costo lineal en k; ampliar el historial exigiría medir memoria y reversión real.

Trazabilidad: logica/historial_reversible.py; logica/cambios.py; logica/motor_juego.py.

4.7 El rastro del jugador

1. Problema

Cada rastreador debe mirar solo sus salidas accesibles y escoger el rastro vigente más reciente, no buscar rutas. En las cargas de 10, 100 y 300 actores, con 4 puertas por sala, se contaron 40, 400 y 1.200 consultas de rastro, una decisión, una comprobación de pertenencia y un retiro por actor. El perfil demo registró 2 actualizaciones de rastro en un movimiento.

2. Alternativas consideradas

Guardar todas las visitas aumenta memoria y obliga a buscar en el historial. Hacer BFS global incumple la regla local. Programar expiraciones añade eventos para limpiar un único entero por sala, sin reducir la consulta. Mantener solo una lista de enemigos implica búsquedas y desplazamientos repetidos y puede volver cuadrático el procesamiento de varios actores.

3. Decisión

Cada sala guarda ultimo_rastro, leído directamente a través de puertas con referencia a su destino. Se aplica caducidad perezosa: $0 \le \text{reloj} - \text{marca} < 400$; el empate usa menor ID. Las salas mantienen tablas hash propias de identidad y posición de enemigos. Para retirar un actor conocido se intercambia con el último y se hace pop del final.

4. Análisis de complejidad

Consultar o actualizar una marca cuesta O(1). Decidir cuesta O(d), con d puertas, y no depende del recorrido del jugador. Para a actores, decidir y retirar cuesta $O(a + \Sigma d_i)$ esperado, excluida la agenda O(log n). Los índices hash tienen costo esperado/amortizado, no garantía constante frente a colisiones adversas. El rastro ocupa O(V) en todo el mapa.

5. Evidencia empírica

ACTORES	CONSULTAS DE RASTRO	DECIDIR + PERTENENCIA + RETIRO
10	40	77,80 µs
100	400	510,90 µs
300	1200	1.444,80 µs

[E/P] Los conteos se instrumentaron fuera del cronómetro. La operación medida llama al comportamiento y a los índices reales con actores sintéticos; no ejecuta el turno completo del motor ni incluye agenda, combate, catálogo, renderizado o movimientos reversibles.

6. Límites conocidos

Un grado muy alto aumenta el trabajo por decisión. El intercambio altera el orden físico de enemigos: es válido porque actúan por la agenda, pero no si una regla futura usa ese orden. El diseño conserva únicamente la última visita y no sirve si se exigen rutas, trayectorias o varias huellas simultáneas.

Trazabilidad: logica/registro_rastro.py; logica/comportamiento_enemigos.py; dto/sala.py.

4.8 Bitácora en pantalla

1. Problema

La interfaz debe conservar solo los últimos 20 mensajes sin mover físicamente los anteriores. Se midieron lotes de 8, 16, 32, 64, 128 y 512 inserciones, una inserción por mensaje. Con 512 mensajes se realizan 20 ocupaciones iniciales y 492 sobrescrituras. No se interpreta esa carga como 512 mensajes generados por una acción habitual.

2. Alternativas consideradas

Una lista sin límite acumula toda la partida y viola el límite de memoria. Una lista de 20 con pop(0) desplaza 19 referencias por descarte, expresamente prohibido. Una lista doble resuelve extremos en O(1), pero asigna nodos de forma innecesaria. deque(maxlen=20) sería válida en rendimiento, pero sustituiría la estructura propia exigida para la evaluación.

3. Decisión

BitacoraPantalla encapsula una ColaCircular de 20 posiciones. Frente, final y tamaño distinguen el orden lógico del físico. Cuando se llena, el siguiente mensaje sobrescribe al más antiguo y los índices avanzan módulo 20. La vista y el controlador comparten la instancia para evitar mensajes duplicados.

4. Análisis de complejidad

Agregar es O(1) estructural, sin desplazamientos y con 20 posiciones permanentes. Consultar reconstruye el orden cronológico en O(k), $k \leq 20$, con una copia auxiliar acotada. El número de referencias es constante; los bytes dependen de la longitud de los 20 textos vigentes. Formatear un mensaje de longitud b puede añadir O(b), fuera del costo del buffer.

5. Evidencia empírica

MENSAJES	COLA CIRCULAR	LISTA CON pop(0)
32	12,40 µs	5,70 µs
128	47,40 µs	24,80 µs
512	205,20 µs	190,80 µs

[B] Se midieron directamente ambas alternativas e incluyeron la conversión de cada número a texto. La lista puede ser más rápida en tiempo absoluto por sus operaciones en C y por el límite fijo 20. No se afirma una victoria temporal del buffer: se elige porque evita desplazar las 19 referencias y reutiliza almacenamiento, como exige el contrato.

6. Límites conocidos

No conserva auditoría completa ni permite buscar mensajes antiguos. Textos individuales ilimitados impiden garantizar bytes constantes aunque haya solo 20. Si se requieren filtros históricos o persistencia, debe añadirse otro registro, sin convertir la bitácora efímera en una colección creciente.

Trazabilidad: estructuras/cola_circular.py; logica/bitacora_pantalla.py; controller/controlador_juego.py.

4.9 Ordenamiento

1. Problema

Inventario, puntajes, catálogo, precarga e índice del guardado requieren ordenar sin destruir el orden original. Cada muestra ordena una colección completa. --bench ensaya 6 tamaños, 4 distribuciones y 2 criterios, con 15 repeticiones y un calentamiento; también calibra factores 1, 2, 4 y 8. Son cargas controladas, no un conteo de consultas de vistas por jugador.

2. Alternativas consideradas

Usar siempre inserción penaliza listas grandes invertidas; usar siempre mezcla desperdicia el orden previo y añade sobrecarga en entradas pequeñas. Elegir solo por n no distingue distribuciones. Quicksort simple no garantiza estabilidad. sorted() y list.sort() se midieron como referencia externa cuando corresponde, pero están prohibidos para estos ordenamientos del proyecto.

3. Decisión

OrdenadorAdaptativo calcula cada clave una vez. Usa inserción estable si $n \leq 16$; para listas mayores intenta inserción con límite n de desplazamientos. Si lo supera, descarta ese intento y aplica mezcla estable reutilizando las claves. Opera sobre copias para no modificar inventario, datos persistidos ni entradas del llamador.

4. Análisis de complejidad

Inserción cuesta $O(n + I)$, con I inversiones, y $O(n^2)$ sin límite en el peor caso. El intento limitado agrega $O(n)$; después mezcla conserva $O(n \log n)$. Entradas ordenadas terminan en $O(n)$; memoria auxiliar $O(n)$. Si calcular una clave cuesta K, se añade $O(nK)$. No se promete que el adaptativo sea el más rápido en cada muestra.

5. Evidencia empírica

CASO / n	INSERCIÓN	MEZCLA	ADAPTATIVO
invertido / 8	6,90 µs	9,40 µs	7,10 µs
invertido / 32	88,40 µs	46,20 µs	61,60 µs
invertido / 512	22.616,00 µs	1.063,30 µs	1.225,70 µs
casi_ordenado / 512	122,30 µs	991,50 µs	192,30 µs

[B] Con n = 512 aleatorio y criterio costoso, el adaptativo tomó 4.502,80 µs. Los casos pequeños y casi ordenados justifican conservar inserción; el crecimiento invertido exige limitarla. El umbral 16 es una elección práctica, no una frontera universal demostrada por una sola ejecución.

6. Límites conocidos

Cambios de Python, hardware, claves o distribuciones requieren recalibrar. Las pruebas de estabilidad y la restricción AST complementan el tiempo. Para volúmenes que no caben en RAM haría falta ordenamiento externo; las copias actuales requieren memoria lineal.

Trazabilidad: logica/ordenamiento.py; benchmarks/integrante3.py; tests/test_restricciones_enunciado.py.

4.10 Guardado binario de la partida

1. Problema

Guardar debe preservar el estado y permitir recuperar una sala sin decodificar todo el archivo, además de rechazar versiones incompatibles. --bench ejecuta por cada una de las 4 criptas una escritura, una carga completa y una lectura de sala por muestra y formato: 15 repeticiones más calentamiento. No hay frecuencia observada de guardado de usuarios.

2. Alternativas consideradas

JSON es legible y carga todo con rapidez, pero requiere parsear el documento para encontrar una sala. Un binario sin índice obliga a recorrer registros. Serializar una tabla hash complica el formato estático; una base embebida añade alcance innecesario. pickle no ofrece un contrato binario propio controlado y es inseguro desde fuentes no confiables.

3. Decisión

Formato propio v7: cabecera fija de 30 bytes con firma CRPT, textos variables, registros y un índice ordenado de 36 bytes por sala. El índice usa OrdenadorAdaptativo. leer_sala lee cabecera, índice y registro objetivo mediante offsets. La escritura temporal, la validación y os.replace protegen el guardado anterior; el historial no se conserva.

4. Análisis de complejidad

Guardar cuesta O(B + V log V), con B bytes. Cargar todo es O(B). Buscar en el índice es O(log V), pero leer_sala completo cuesta O(V + r): lee todo el índice y lo recorre para hallar el fin del registro de r bytes. Memoria auxiliar O(V + r) para esa lectura. No se afirma acceso total O(log V).

5. Evidencia empírica

CRIPTA / SALAS	BYTES BIN. / JSON	1 SALA · BIN. / JSON	TODO · BIN. / JSON
cripta-01 / 6	5.359 / 12.709	233,30 / 371,70 µs	484,40 / 341,00 µs
cripta-04 / 50	17.008 / 43.328	234,40 / 686,50 µs	1.273,50 / 634,40 µs

[B] Fuente de la tabla: docs/mediciones_entrega.json, Windows 11 y Python 3.14.0. Los bytes coinciden con docs/comparacion_binario_json.md; los tiempos pertenecen a esta ejecución, no a la serie anterior de ese archivo. El binario favorece tamaño y lectura parcial; JSON gana en carga completa. La instalación de Windows usa zlib-ng: comprime el azar en 3.375 bytes, frente a 3.573 con zlib clásico. Esto explica los 198 bytes adicionales de WSL sin cambiar el formato v7 ni el estado recuperado.

6. Límites conocidos

El índice crece linealmente y los offsets son de 32 bits. Para archivos grandes convendría almacenar longitudes y leer entradas del índice bajo demanda. Cambiar el esquema exige otra versión. Las mediciones usan archivos locales temporales; no incluyen OneDrive ni latencia de almacenamiento remoto.

Trazabilidad: datos/guardado_binario.py; benchmarks/integrante2.py; service/juego_service.py.

4.11 Bitácora de partida y reproducción

1. Problema

Se necesita reconstruir una partida desde decisiones, semilla y versiones, sin reescribir el historial en cada acción. El proveedor adicional midió un anexado real con 32, 512 y 4.096 acciones previas y la validación completa de cada archivo. El proveedor original parsea lotes de 8 a 512 acciones; no ejecuta la simulación completa del replay.

2. Alternativas consideradas

Un arreglo JSON único permite parseo conjunto más rápido, pero agregar exige editar el cierre o reescribirlo, en lugar de anexar una unidad independiente. CSV o texto con espacios requiere convenciones de escape. El binario dificulta inspeccionar reportes y no cumple el requisito textual. Nombres visibles como objetivos pueden repetirse; se descartan frente a IDs estables.

3. Decisión

RegistroPartida escribe JSON Lines UTF-8: cabecera con cripta, versiones y semilla; después una línea por acción exitosa con IDs. La creación es exclusiva y el anexado usa modo a. El controlador inicia el registro automáticamente cuando la partida es reproducible. EjecutorReplay valida todo antes de ejecutar y rechaza versiones no disponibles; utiliza el mismo servicio y motor.

4. Análisis de complejidad

Anexar serializa y escribe O(b) bytes, sin leer el log previo. Validar cuesta O(L) tiempo y memoria proporcional a las acciones/textos retenidos, con L bytes del archivo. Reproducir añade el costo de cada acción, sus eventos y resolver objetivos O(e) u O(i). No basta con citar el parseo para estimar todo el replay.

5. Evidencia empírica

ACCIONES PREVIAS	ANEXAR 1 ACCIÓN	VALIDAR LOG COMPLETO
32	0,48 ms	0,50 ms
512	0,52 ms	3,81 ms
4096	0,49 ms	31,35 ms

[B/E] Para 512 acciones, solo parsear JSON Lines tomó 1.203,90 µs frente a 352,70 µs del arreglo JSON descartado. Esa ventaja de lectura no resuelve el anexado. La medición real incluye abrir/cerrar archivo, pero no fsync ni el resto de comprobaciones de JuegoService.

6. Límites conocidos

Validar y almacenar el archivo completo consume memoria lineal: logs extensos aconsejan dos pasadas. Datos/versiones o reglas distintas impiden garantizar fidelidad. El registro de acciones no equivale a un guardado retomado; deben declararse las limitaciones de sesiones cargadas y demo.

Trazabilidad: datos/registro_partida.py; controller/ejecutor_replay.py; service/juego_service.py.

Síntesis y condiciones de revisión

La selección no maximiza una única métrica. La agenda favorece operaciones repetidas y cancelación dirigida; la carga y el catálogo reducen consultas; el inventario y la cola circular satisfacen restricciones estructurales; el historial conserva cambios mínimos; y el guardado y la reproducción responden a dos necesidades distintas: acceso al estado y repetición de decisiones.

DECISIÓN	RAZÓN DETERMINANTE	CUÁNDO REEVALUAR
Agenda indexada	Mínimo frecuente y cancelación por referencia	Gran volumen o predominio de mutaciones
Precarga por cercanía	Disponibilidad antes de entrar y presupuesto	Mapas enormes o latencia concurrente
Catálogo por lotes	Reutilización de tipos y control de solicitudes	Muchos tipos o botín transitivo extenso
LRU acotada	Localidad sin retener fichas en entidades	Conjunto activo mayor que la capacidad
Lista doble + cursor	Operar sobre la selección sin desplazar	Predominio de búsquedas por ID
Transacciones inversas	Deshacer sin instantáneas del estado	Intervalos con muchísimos cambios
Rastro por sala	Decisión estrictamente local	Rutas o varias huellas por sala
Cola circular	20 posiciones; sin desplazar mensajes	Auditoría persistente o textos ilimitados
Ordenamiento adaptativo	Tamaño e inversiones observadas	Otra plataforma o costo de claves
Binario indexado	Menor tamaño y lectura parcial	Índices muy grandes o límite de offsets
JSON Lines	Anexado independiente e inspección	Logs que no caben en memoria

Conclusión

Los resultados respaldan las decisiones dentro de las cargas descritas y hacen visibles sus costos. No demuestran que toda alternativa descartada sea siempre más lenta. Se conservan explícitamente los límites: validación reversible no necesariamente O(k), lectura parcial no totalmente O(log V), LRU con trabajo adicional por fijaciones y frecuencias propias de escenarios controlados.

Una solución sencilla se considera suficiente solo para una escala respaldada por datos y si conserva el contrato al crecer, mediante límites independientes del mapa o adaptación ya implementada. Esta condición se verifica por decisión, no se deduce de una prueba pequeña. El anexo C separa los casos sustentados de las garantías todavía pendientes.

Deben repetirse las mediciones si cambian reglas, escalas, formato, umbrales o plataforma. Los proveedores E y perfiles P ya están integrados en python main.py --bench y aparecen en docs/mediciones_entrega.json. Se resuelve así la dependencia del ejecutor externo para §8.1 y §8.5; los límites de escala, alcance y rendimiento global declarados siguen vigentes.

Anexo A · Evidencia y reproducción

Resultados incluidos en el documento

Las tablas se actualizan con docs/mediciones_entrega.json, una ejecución de python main.py --bench en Windows 11 y Python 3.14.0. Contiene 633 registros: 607 de proveedores originales B, 21 mediciones complementarias E y 5 perfiles P. Las seis estimaciones de red presentes en B proceden del enunciado, no de mediciones HTTP.

Ejecución estándar

Desde la carpeta JuegoCripta:

python main.py --bench > docs/mediciones_entrega.json

Mediciones complementarias incluidas en --bench

main.py concatena los proveedores originales con PROVEEDORES de benchmarks/bench_ampliado.py. El comando estándar ejecuta precarga, catálogo, rastro, inventario, replay y frecuencias sin un paso externo. Para reproducir, ejecutar el comando anterior desde JuegoCripta con sus dependencias y datos offline; comprobar el entorno y las claves del JSON antes de comparar tiempos.

Protocolo: 15 muestras y un calentamiento por caso temporal, preparación fuera del cronómetro y mediana obtenida con perf_counter_ns, salvo agenda y ciclo, que usan perf_counter. El ciclo demo usa 200 muestras y 10 calentamientos. Los perfiles P se recogen una vez por escenario, separados del cronómetro.

Se probaron cuadrículas de 100, 400 y 900 salas; resolución de 32, 128 y 512 tipos en memoria; y 10, 100 y 300 rastreadores con cuatro puertas por sala. También se midieron retiros en tres posiciones del inventario y anexado/validación de registros. Los detalles y límites de cada carga se describen en sus respectivas secciones.

Correspondencia entre resultados y secciones

SECCIÓN	CLAVES DE MEDICIÓN EN EL JSON
4.1	agenda_extraer_eventos; ciclo_simulacion_demo; perfil_frecuencia_ciclo_demo
4.2	precarga_bfs_*; perfil_arranque_offline
4.3	catalogo_resolver_lotes_memoria; perfil_arranque_offline
4.4	cache_buscar_*; cache_lista_lineal_*; cache_recuperar_desde_disco; perfil_arranque_offline
4.5	inventario_*; lista_python_retirar_frente
4.6	historial_*; perfil_frecuencia_ciclo_demo
4.7	rastro_decidir_pertenencia_retirar; perfil_frecuencia_ciclo_demo
4.8	bitacora_circular; bitacora_lista_alternativa
4.9	ordenar; calibracion_factor
4.10	guardado_binario_*; guardado_json_*
4.11	replay_parse_*; replay_anexar_accion_real; replay_validar_log_real

Unidades: mediana original en ns_por_lote; 1 µs = 1.000 ns y 1 ms = 1.000.000 ns. Las tablas redondean a dos decimales. Los perfiles expresan conteos y no tiempos.

Anexo B · Fuentes y trazabilidad

Documentación consultada

1. estructuras_utilizadas.md. Base para identificar las decisiones §4.1–§4.11.

2. docs/enunciado.md del proyecto: contratos de juego §2, servicio §3, problemas §4, rendimiento §5, restricciones §7 y requisitos del documento §8.

3. Código de producción y benchmarks de JuegoCripta, junto con docs/mediciones_entrega.json y docs/comparacion_binario_json.md. Las claves de resultados y las notas de trazabilidad identifican los métodos utilizados. Los tiempos de las tablas proceden del JSON de entrega; la comparación escrita aporta contexto del formato y del compresor.

Proveedores originales

benchmarks/integrante3.py: inventario, historial, cola circular, ordenamiento, parseo de replay y agregación de proveedores. benchmarks/integrante2.py: caché, recuperación desde disco, binario frente a JSON y tabla hash. benchmarks/benchmark_agenda_integrante1.py y prueba_rendimiento_sofia.py: extracción y ciclo demo.

Proveedores complementarios integrados

benchmarks/bench_ampliado.py exporta proveedores para BFS frente a listas, resolución local de catálogo, rastreadores, retiros de inventario, anexado y validación de registros. Incluye perfiles de arranque offline y ciclo demo. Todos están conectados a python main.py --bench. Sus dobles sintéticos no sustituyen estructuras de producción ni miden latencia HTTP.

Frecuencias: lectura correcta

El arranque de cada cripta se cuenta una vez en condiciones frías. El ciclo demo se instrumenta una vez; su tiempo se mide en otras 200 ejecuciones sin perfilador. medir_frecuencias forma parte del comando estándar y genera los cinco perfiles del JSON. Estos conteos no describen una partida promedio ni justifican mezclar lotes distintos para calcular operaciones por segundo.

Verificación de corrección adicional

Se ejecutaron tests/test_service/test_cargador_bfs.py, tests/test_carga_gradual.py, tests/test_bench_integrante2.py y tests/test_restricciones_enunciado.py: 64 pruebas aprobadas. No se presentan como una ejecución de toda la suite ni se reutilizan los conteos históricos del archivo base como si fueran de esta revisión.

Alcance de la evidencia

Las tablas usan una única ejecución Windows: docs/mediciones_entrega.json. Los tiempos históricos de WSL y los de otras corridas documentadas no se atribuyen a esa ejecución. E/P están incluidos en el comando estándar y verificados en el JSON de entrega. Esto resuelve su reproducción, no certifica ciclos completos de criptas mayores ni elimina los límites técnicos descritos.

Documento elaborado con apoyo de IA a partir de fuentes locales y ejecuciones verificables. La revisión final y la defensa de las decisiones corresponden al equipo del proyecto.

Anexo C · Verificación del criterio de escala

Criterio aplicado

“Concluir que una alternativa sencilla es suficiente para cierta escala es válido si las mediciones lo sustentan y el programa sigue cumpliendo los requisitos cuando la cripta crece”. No basta con decir “hay pocos elementos”: se exige evidencia para esa escala y un mecanismo implementado que conserve las propiedades requeridas al aumentar la carga.

Casos con justificación sustentada

DECISIÓN	EVIDENCIA PROPIA	QUÉ CONSERVA AL CRECER
Inserción para colecciones pequeñas (§4.9)	Invertidos, n = 8: inserción 6,90 µs; mezcla 9,40 µs.	Con n > 16 se limita el intento a n desplazamientos y se pasa a mezcla si es necesario: no se conserva inserción cuadrática sin límite.
Cambio automático de algoritmo (§4.9)	Invertidos, n = 512: inserción 22,62 ms; adaptativo 1,23 ms.	La alternativa sencilla no se usa incondicionalmente. Se mantienen estabilidad, entrada sin mutación y peor caso O(n log n).
Buffer fijo de 20 mensajes (§4.8)	512 mensajes: 205,20 µs por lote; las pruebas verifican capacidad y sobrescritura.	Más salas o mensajes no agrandan el arreglo ni provocan desplazar los 19 anteriores. El tamaño de cada texto sigue siendo una salvedad.
Precarga con pertenencia hash (§4.2)	900 salas: hash 9,57 ms frente a listas 24,97 ms; mismo lote.	Se evita pertenencia lineal repetida. La planificación total aún incluye O(V log V) por ordenar y no es una garantía del ciclo completo.

Dónde la condición aún no está demostrada

Catálogo (§4.3): los 11 tipos de las criptas publicadas no justifican afirmar suficiencia para muchos tipos nuevos; las listas de vistos pueden ser cuadráticas y no existe un reemplazo automático implementado. Caché (§4.4): si el conjunto en uso supera la capacidad, se rechazan inserciones; no está demostrado cómo satisfacer simultáneamente retención de todas las fichas activas y tope en ese caso.

Rendimiento global (§5.2): los 200 ciclos demo inferiores a 20 ms no certifican ciclos completos con cientos de enemigos y efectos. Medir subsistemas por separado tampoco basta. Haría falta ejecutar escenarios completos de mayor escala y comprobar todas las restricciones, incluido el presupuesto de solicitudes.

Dictamen de la revisión

La justificación de sencillez por escala es válida en los casos identificados, especialmente el ordenamiento adaptativo. No se emite una certificación global de cumplimiento del programa para cualquier crecimiento. Las mejoras sugeridas son pendientes, no estrategias ya ejecutadas. Se aprobaron 124 pruebas dirigidas adicionales de BFS, rastro, caché, retroceso, restricciones, ordenamiento y cola circular; son evidencia de corrección de los casos cubiertos, no garantía universal de rendimiento.